

TOP 100 REAL-WORLD SQL PROBLEM-SOLVING QUESTIONS WITH ANSWERS

MySQL 8+ • Data Analyst & SQL Interview Preparation

Created by Aditya Kalyan | Data Analyst Mentor

Data Detective Club

QUESTION DISTRIBUTION

Questions	Section
Q01–Q15	SQL Fundamentals
Q16–Q25	Filtering & Aggregation
Q26–Q40	Joins
Q41–Q50	Subqueries & CTEs
Q51–Q65	Window Functions
Q66–Q75	Date & Time Analysis
Q76–Q85	Real-World Data Analytics
Q86–Q95	Advanced SQL
Q96–Q100	Hard Interview Challenges

Q01. Find all employees

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q02. Select employee names and salaries

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT name, salary FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q03. Find employees earning more than 60000

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees WHERE salary > 60000;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q04. Find employees with salary between 49000 and 75000

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees WHERE salary BETWEEN 49000 AND 75000;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q05. Sort employees by salary descending

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees ORDER BY salary DESC;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q06. Find unique departments

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT DISTINCT department FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q07. Count employees

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT COUNT(*) AS employee_count FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q08. Find average salary

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q09. Find highest salary

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT MAX(salary) AS highest_salary FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q10. Find lowest salary

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT MIN(salary) AS lowest_salary FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q11. Find total salary expense

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT SUM(salary) AS total_salary FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q12. Count employees by department

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q13. Average salary by department

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q14. Departments with average salary above 70000

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department HAVING AVG(salary) > 70000;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q15. Classify salary using CASE

Section: SQL Fundamentals **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT name, salary, CASE WHEN salary >= 80000 THEN 'High' WHEN salary >= 50000 THEN 'Medium' ELSE 'Low'  
END AS salary_band FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q16. Employees with department details

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT e.name, d.department_name FROM employees e INNER JOIN departments d ON  
e.department_id=d.department_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q17. All employees including those without a department

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT e.name, d.department_name FROM employees e LEFT JOIN departments d ON  
e.department_id=d.department_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q18. Departments with no employees

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT d.department_name FROM departments d LEFT JOIN employees e ON d.department_id=e.department_id  
WHERE e.employee_id IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q19. Employees earning above 60000 with department

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT e.name, e.salary, d.department_name FROM employees e JOIN departments d ON  
e.department_id=d.department_id WHERE e.salary>60000;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q20. Count employees in each department

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT d.department_name, COUNT(e.employee_id) AS employee_count FROM departments d LEFT JOIN employees e  
ON d.department_id=e.department_id GROUP BY d.department_name;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q21. Find managers and employees

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT e.name AS employee, m.name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id=m.employee_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q22. Employees earning more than their manager

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS manager_salary FROM employees e JOIN employees m ON e.manager_id=m.employee_id WHERE e.salary>m.salary;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q23. Customers who never ordered

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT c.customer_id,c.name FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q24. Orders with customer names

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT o.order_id,c.name,o.order_date,o.amount FROM orders o JOIN customers c ON o.customer_id=c.customer_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q25. Top 3 salaries overall

Section: Filtering & Aggregation **Difficulty:** Beginner

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q26. Second-highest salary

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT MAX(salary) AS second_highest FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q27. Nth-highest salary using DENSE_RANK

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT name,salary FROM (SELECT name,salary,DENSE_RANK() OVER(ORDER BY salary DESC) rn FROM employees) x WHERE rn=3;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q28. Rank employees by salary

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT name,salary,RANK() OVER(ORDER BY salary DESC) AS salary_rank FROM employees;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q29. Top 3 earners per department

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT name,department,salary FROM (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) rn FROM employees e) x WHERE rn<=3;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q30. Highest salary per department

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary DESC) rn FROM employees e) x WHERE rn=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q31. Lowest salary per department

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT e.*,ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary) rn FROM employees e) x WHERE rn=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q32. Running total of sales

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT order_date,amount,SUM(amount) OVER(ORDER BY order_date) AS running_total FROM orders;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q33. Monthly running revenue

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT DATE_FORMAT(order_date,'%Y-%m') month,SUM(amount) revenue,SUM(SUM(amount)) OVER(ORDER BY DATE_FORMAT(order_date,'%Y-%m')) running_revenue FROM orders GROUP BY DATE_FORMAT(order_date,'%Y-%m');
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q34. Previous order amount

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,order_date,amount,LAG(amount) OVER(PARTITION BY customer_id ORDER BY order_date) AS previous_amount FROM orders;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q35. Next order amount

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,order_date,amount,LEAD(amount) OVER(PARTITION BY customer_id ORDER BY order_date) AS next_amount FROM orders;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q36. Days between customer orders

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,order_date,DATEDIFF(order_date,LAG(order_date) OVER(PARTITION BY customer_id ORDER BY order_date)) days_since_previous FROM orders;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q37. Duplicate emails

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT email,COUNT(*) cnt FROM customers GROUP BY email HAVING COUNT(*)>1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q38. Remove duplicate rows conceptually

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT email,MIN(customer_id) keep_id FROM customers GROUP BY email HAVING COUNT(*)>1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q39. Customers with at least 3 orders

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,COUNT(*) order_count FROM orders GROUP BY customer_id HAVING COUNT(*)>=3;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q40. Products sold more than 100 units

Section: Joins **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT product_id,SUM(quantity) units FROM order_items GROUP BY product_id HAVING SUM(quantity)>100;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q41. Best-selling product

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT product_id,SUM(quantity) units FROM order_items GROUP BY product_id ORDER BY units DESC LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q42. Revenue by product

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT product_id,SUM(quantity*unit_price) revenue FROM order_items GROUP BY product_id ORDER BY revenue DESC;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q43. Revenue by month

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') month,SUM(amount) revenue FROM orders GROUP BY month ORDER BY month;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q44. Orders in last 30 days

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q45. Orders this year

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT * FROM orders WHERE YEAR(order_date)=YEAR(CURRENT_DATE);
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q46. Monthly customer count

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') month,COUNT(DISTINCT customer_id) customers FROM orders GROUP BY month;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q47. Customers first order date

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,MIN(order_date) first_order_date FROM orders GROUP BY customer_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q48. Latest order per customer

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT o.*,ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date DESC) rn FROM orders o) x WHERE rn=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q49. Month-over-month revenue growth

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT month,revenue,LAG(revenue) OVER(ORDER BY month) prev_revenue,ROUND(100*(revenue-LAG(revenue) OVER(ORDER BY month))/NULLIF(LAG(revenue) OVER(ORDER BY month),0),2) growth_pct FROM monthly_sales;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q50. Year-over-year revenue

Section: Subqueries & CTEs **Difficulty:** Intermediate

Task: Solve the business/data problem using SQL.

```
SELECT year,revenue,LAG(revenue) OVER(ORDER BY year) prev_year_revenue FROM yearly_sales;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q51. Employees joined in 2026

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT * FROM employees WHERE YEAR(join_date)=2026;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q52. Users active on at least 3 different days

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT user_id FROM user_logins GROUP BY user_id HAVING COUNT(DISTINCT DATE(login_time))>=3;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q53. Users with consecutive login days

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH d AS (SELECT DISTINCT user_id,DATE(login_time) dt FROM user_logins), x AS (SELECT user_id,dt,LAG(dt) OVER(PARTITION BY user_id ORDER BY dt) prev_dt FROM d) SELECT DISTINCT user_id FROM x WHERE DATEDIFF(dt,prev_dt)=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q54. Users with no consecutive login days

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH d AS (SELECT DISTINCT user_id,DATE(login_time) dt FROM user_logins), x AS (SELECT user_id,dt,LAG(dt)
OVER(PARTITION BY user_id ORDER BY dt) prev_dt FROM d) SELECT user_id FROM d GROUP BY user_id HAVING
SUM(CASE WHEN DATEDIFF(dt,prev_dt)=1 THEN 1 ELSE 0 END)=0;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q55. Above-average customer spend

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,SUM(amount) spend FROM orders GROUP BY customer_id HAVING SUM(amount)>(SELECT
AVG(total_spend) FROM (SELECT customer_id,SUM(amount) total_spend FROM orders GROUP BY customer_id) s);
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q56. Customers who bought every product

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT customer_id FROM order_items GROUP BY customer_id HAVING COUNT(DISTINCT product_id)=(SELECT
COUNT(*) FROM products);
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q57. Products never sold

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT p.product_id,p.product_name FROM products p LEFT JOIN order_items oi ON p.product_id=oi.product_id
WHERE oi.product_id IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q58. Category revenue

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT p.category,SUM(oi.quantity*oi.unit_price) revenue FROM products p JOIN order_items oi ON
p.product_id=oi.product_id GROUP BY p.category;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q59. Top category by revenue

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT category,revenue FROM (SELECT p.category,SUM(oi.quantity*oi.unit_price) revenue FROM products p
JOIN order_items oi ON p.product_id=oi.product_id GROUP BY p.category) x ORDER BY revenue DESC LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q60. Average order value

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT AVG(order_total) avg_order_value FROM (SELECT order_id,SUM(quantity*unit_price) order_total FROM
order_items GROUP BY order_id) x;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q61. Customers with orders in two consecutive days

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH d AS (SELECT DISTINCT customer_id,DATE(order_date) dt FROM orders), x AS (SELECT customer_id,dt,LAG(dt) OVER(PARTITION BY customer_id ORDER BY dt) prev_dt FROM d) SELECT DISTINCT customer_id FROM x WHERE DATEDIFF(dt,prev_dt)=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q62. Customer retention by month

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH firsts AS (SELECT customer_id,DATE_FORMAT(MIN(order_date),'%Y-%m') cohort FROM orders GROUP BY customer_id), activity AS (SELECT DISTINCT customer_id,DATE_FORMAT(order_date,'%Y-%m') month FROM orders) SELECT f.cohort,a.month,COUNT(DISTINCT a.customer_id) retained_users FROM firsts f JOIN activity a ON f.customer_id=a.customer_id GROUP BY f.cohort,a.month;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q63. Churned customers

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT c.customer_id FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id GROUP BY c.customer_id HAVING MAX(o.order_date)<CURRENT_DATE-INTERVAL 90 DAY;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q64. Funnel step conversion

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT COUNT(DISTINCT CASE WHEN event_name='visit' THEN user_id END) visits,COUNT(DISTINCT CASE WHEN event_name='signup' THEN user_id END) signups,COUNT(DISTINCT CASE WHEN event_name='purchase' THEN user_id END) purchases FROM events;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q65. Purchase conversion rate

Section: Window Functions **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT ROUND(100*COUNT(DISTINCT CASE WHEN event_name='purchase' THEN user_id END)/NULLIF(COUNT(DISTINCT CASE WHEN event_name='visit' THEN user_id END),0),2) conversion_pct FROM events;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q66. Daily active users

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT DATE(event_time) dt,COUNT(DISTINCT user_id) dau FROM events WHERE event_name='login' GROUP BY DATE(event_time);
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q67. Monthly active users

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT DATE_FORMAT(event_time, '%Y-%m') month, COUNT(DISTINCT user_id) mau FROM events GROUP BY month;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q68. DAU/MAU ratio

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH d AS (SELECT DATE(event_time) dt, COUNT(DISTINCT user_id) dau FROM events GROUP BY DATE(event_time)),  
m AS (SELECT DATE_FORMAT(event_time, '%Y-%m') month, COUNT(DISTINCT user_id) mau FROM events GROUP BY  
month) SELECT DATE_FORMAT(d.dt, '%Y-%m') month, AVG(d.dau)/MAX(m.mau) dau_mau_ratio FROM d JOIN m ON  
DATE_FORMAT(d.dt, '%Y-%m')=m.month GROUP BY DATE_FORMAT(d.dt, '%Y-%m');
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q69. Median salary

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT AVG(salary) median_salary FROM (SELECT salary, ROW_NUMBER() OVER(ORDER BY salary) rn, COUNT(*)  
OVER() cnt FROM employees) x WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2));
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q70. Salary percentile

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT salary FROM (SELECT salary, PERCENT_RANK() OVER(ORDER BY salary) pct FROM employees) x WHERE  
pct>=0.9 ORDER BY pct LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q71. Gap between salary ranks

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT name, salary, salary-LAG(salary) OVER(ORDER BY salary DESC) gap_from_previous FROM employees ORDER  
BY salary DESC;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q72. Three-month moving average

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT month, revenue, AVG(revenue) OVER(ORDER BY month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)  
moving_avg FROM monthly_sales;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q73. First purchase in each category

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT oi.customer_id,p.category,o.order_date,ROW_NUMBER() OVER(PARTITION BY
oi.customer_id,p.category ORDER BY o.order_date) rn FROM order_items oi JOIN products p ON
oi.product_id=p.product_id JOIN orders o ON oi.order_id=o.order_id) x WHERE rn=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q74. Repeat customers

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*)>1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q75. One-time customers

Section: Date & Time Analysis **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*)=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q76. Customers with increasing order values

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH x AS (SELECT customer_id,order_date,amount,LAG(amount) OVER(PARTITION BY customer_id ORDER BY
order_date) prev_amount FROM orders) SELECT DISTINCT customer_id FROM x WHERE prev_amount IS NOT NULL AND
amount>prev_amount;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q77. Declining monthly sales

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH x AS (SELECT month,revenue,LAG(revenue) OVER(ORDER BY month) prev_revenue FROM monthly_sales) SELECT
month,revenue FROM x WHERE revenue<prev_revenue;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q78. Longest user login streak

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH d AS (SELECT DISTINCT user_id,DATE(login_time) dt FROM user_logins), g AS (SELECT
user_id,dt,DATE_SUB(dt,INTERVAL ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY dt) DAY) grp FROM d)
SELECT user_id,COUNT(*) streak_days FROM g GROUP BY user_id,grp ORDER BY streak_days DESC LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q79. Missing dates in sales

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH RECURSIVE dates AS (SELECT MIN(order_date) dt FROM orders UNION ALL SELECT dt+INTERVAL 1 DAY FROM
dates WHERE dt<(SELECT MAX(order_date) FROM orders)) SELECT dt FROM dates d LEFT JOIN (SELECT DISTINCT
order_date FROM orders) o ON d.dt=o.order_date WHERE o.order_date IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q80. Top 2 products per category by revenue

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT p.category,p.product_id,SUM(oi.quantity*oi.unit_price) revenue,ROW_NUMBER()
OVER(PARTITION BY p.category ORDER BY SUM(oi.quantity*oi.unit_price) DESC) rn FROM products p JOIN
order_items oi ON p.product_id=oi.product_id GROUP BY p.category,p.product_id) x WHERE rn<=2;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q81. Employees with same salary

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT e1.name,e1.salary,e2.name duplicate_name FROM employees e1 JOIN employees e2 ON
e1.salary=e2.salary AND e1.employee_id<e2.employee_id;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q82. Departments with more than 5 employees

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT department,COUNT(*) cnt FROM employees GROUP BY department HAVING COUNT(*)>5;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q83. Customers spending more than 100000

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,SUM(amount) spend FROM orders GROUP BY customer_id HAVING SUM(amount)>100000;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q84. Product contribution to category revenue

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
WITH p AS (SELECT pr.category,oi.product_id,SUM(oi.quantity*oi.unit_price) revenue FROM products pr JOIN
order_items oi ON pr.product_id=oi.product_id GROUP BY pr.category,oi.product_id), c AS (SELECT
category,SUM(revenue) total_revenue FROM p GROUP BY category) SELECT
p.category,p.product_id,p.revenue,ROUND(100*p.revenue/c.total_revenue,2) contribution_pct FROM p JOIN c
ON p.category=c.category;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q85. Find orders with duplicate order amounts

Section: Real-World Data Analytics **Difficulty:** Advanced

Task: Solve the business/data problem using SQL.

```
SELECT amount,COUNT(*) cnt FROM orders GROUP BY amount HAVING COUNT(*)>1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q86. Detect price changes

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT product_id,effective_date,price,LAG(price) OVER(PARTITION BY product_id ORDER BY effective_date)
previous_price FROM product_prices;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q87. Latest price per product

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT * FROM (SELECT pp.*,ROW_NUMBER() OVER(PARTITION BY product_id ORDER BY effective_date DESC) rn
FROM product_prices pp) x WHERE rn=1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q88. Find inactive products

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT p.product_id,p.product_name FROM products p LEFT JOIN order_items oi ON p.product_id=oi.product_id
LEFT JOIN orders o ON oi.order_id=o.order_id GROUP BY p.product_id,p.product_name HAVING
MAX(o.order_date)<CURRENT_DATE-INTERVAL 180 DAY OR MAX(o.order_date) IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q89. Top customer by lifetime value

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT customer_id,SUM(amount) lifetime_value FROM orders GROUP BY customer_id ORDER BY lifetime_value
DESC LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q90. Customers whose latest order is their largest

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH x AS (SELECT o.*,MAX(amount) OVER(PARTITION BY customer_id) max_amount,ROW_NUMBER() OVER(PARTITION
BY customer_id ORDER BY order_date DESC) rn FROM orders o) SELECT customer_id,amount FROM x WHERE rn=1
AND amount=max_amount;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q91. Find missing employee IDs

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH RECURSIVE ids AS (SELECT MIN(employee_id) id FROM employees UNION ALL SELECT id+1 FROM ids WHERE
id<(SELECT MAX(employee_id) FROM employees)) SELECT ids.id FROM ids LEFT JOIN employees e ON
ids.id=e.employee_id WHERE e.employee_id IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q92. Find overlapping date ranges

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT a.id,b.id FROM bookings a JOIN bookings b ON a.id<b.id AND a.start_date<=b.end_date AND b.start_date<=a.end_date;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q93. Customers who bought in every month of a year

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT customer_id FROM orders WHERE YEAR(order_date)=2026 GROUP BY customer_id HAVING COUNT(DISTINCT MONTH(order_date))=12;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q94. Three consecutive months of purchases

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH m AS (SELECT DISTINCT customer_id,EXTRACT(YEAR_MONTH FROM order_date) ym FROM orders), x AS (SELECT customer_id,ym,LAG(ym,1) OVER(PARTITION BY customer_id ORDER BY ym) p1,LAG(ym,2) OVER(PARTITION BY customer_id ORDER BY ym) p2 FROM m) SELECT DISTINCT customer_id FROM x WHERE ym=p1+1 AND p1=p2+1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q95. Customers who purchased category A and B

Section: Advanced SQL **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT customer_id FROM order_items oi JOIN products p ON oi.product_id=p.product_id WHERE p.category IN ('A','B') GROUP BY customer_id HAVING COUNT(DISTINCT p.category)=2;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q96. Most common product pair

Section: Hard Interview Challenges **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH pairs AS (SELECT a.product_id p1,b.product_id p2,a.order_id FROM order_items a JOIN order_items b ON a.order_id=b.order_id AND a.product_id<b.product_id) SELECT p1,p2,COUNT(*) pair_count FROM pairs GROUP BY p1,p2 ORDER BY pair_count DESC LIMIT 1;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q97. Find customers with no purchase after signup

Section: Hard Interview Challenges **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
SELECT c.customer_id FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id AND o.order_date>=c.signup_date WHERE o.order_id IS NULL;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q98. Revenue share by month

Section: Hard Interview Challenges **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH m AS (SELECT DATE_FORMAT(order_date, '%Y-%m') month, SUM(amount) revenue FROM orders GROUP BY month)
SELECT month, revenue, ROUND(100*revenue/SUM(revenue) OVER(), 2) overall_share_pct FROM m;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q99. Top 5% customers by spend

Section: Hard Interview Challenges **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH s AS (SELECT customer_id, SUM(amount) spend FROM orders GROUP BY customer_id), r AS (SELECT
s.*, PERCENT_RANK() OVER(ORDER BY spend DESC) pct FROM s) SELECT * FROM r WHERE pct <= 0.05;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

Q100. Median order value by month

Section: Hard Interview Challenges **Difficulty:** Hard

Task: Solve the business/data problem using SQL.

```
WITH x AS (SELECT DATE_FORMAT(order_date, '%Y-%m') month, amount, ROW_NUMBER() OVER(PARTITION BY
DATE_FORMAT(order_date, '%Y-%m') ORDER BY amount) rn, COUNT(*) OVER(PARTITION BY
DATE_FORMAT(order_date, '%Y-%m')) cnt FROM orders) SELECT month, AVG(amount) median_order_value FROM x
WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2)) GROUP BY month;
```

■ **Interview Tip:** Explain your logic, edge cases, NULL handling, and why your chosen approach is appropriate.

SQL INTERVIEW READINESS CHECKLIST

- SELECT, WHERE, ORDER BY, DISTINCT, GROUP BY, HAVING
- Aggregate functions and conditional aggregation
- INNER, LEFT, RIGHT, FULL and SELF JOIN patterns
- Subqueries, correlated subqueries and CTEs
- ROW_NUMBER, RANK, DENSE_RANK, LAG and LEAD
- Running totals, moving averages and top-N per group
- Date/time analysis, MoM/YoY growth and consecutive-day problems
- Duplicates, NULLs, missing records and gaps & islands
- Customer, sales, product, revenue, retention and churn analysis
- Ability to explain query logic clearly in an interview

7-DAY SQL REVISION PLAN

Day 1 — SQL fundamentals, filtering, sorting, NULLs and constraints.

Day 2 — GROUP BY, HAVING, CASE, aggregates and business KPIs.

Day 3 — Joins, self joins and multi-table analysis.

Day 4 — Subqueries and CTEs; rewrite simple queries using both.

Day 5 — Window functions: ranking, LAG/LEAD, running totals and moving averages.

Day 6 — Dates, retention, cohort, churn, funnels and consecutive-day problems.

Day 7 — Solve 10 mixed hard problems under interview time limits and explain every solution aloud.

Created by Aditya Kalyan | Data Analyst Mentor

Data Detective Club